المدرسة الوطنية للعلوم التطبيقية
ⵜⵉⵎⵙⵙⵓⵜ ⵜⴰⵏⴰⵎⵓⵔⵜ ⵏ ⵜⵙⵙⴰⵏⵉⵡⵉⵏ ⵜⵉⴷⵔⴰⵙⵉⵏ
Ecole Nationale des Sciences Appliquées de Fès

جامعة سيدي محمد بن عبد الله
ⵜⴰⵙⴷⴰⵡⵜ ⵙⵉⴷⵉ ⵎⵓⵃⴰⵎⵎⴷ ⴱⵏ ⵄⴱⴷⵍⵍⴰⵀ
Université Sidi Mohamed Ben Abdellah

TP3 :

AWS Academy Cloud Foundations





BELHAJ KHAOULA

N 9

GSCSI- G1

Table des matières

Table des matières 2
Table de figures 3
Atelier 5 : Création de votre serveur de base de données et interaction avec votre base de données via une application 4
Introduction 4
Tâche 1 : Création d'un groupe de sécurité pour l'instance de base de données RDS 5
Analyse de la configuration : 5
Résumé technique de l'implémentation : 6
Tâche 2 : Création d'un groupe de sous-réseaux de base de données 7
Analyse de la configuration : 7
Résumé technique de l'implémentation : 8
Tâche 3 : Création d'une instance de base de données Amazon RDS 9
Analyse de la configuration : 9
Résumé technique de l'implémentation : 10
Tâche 4 : Interaction avec votre base de données 11
Analyse du processus d'intégration : 11
Importance de la synchronisation : 12
Conclusion 13

Table de figures

Figure 1 Créer un groupe de sécurité 6
Figure 2 Créer un groupe de sous-réseaux DB 8
Figure 3 Créer une base de données 10
Figure 4 WebServer 11
Figure 5 Address Book 12
Figure 6 Add Contact 12

Atelier 5 : Création de votre serveur de base de données et interaction avec votre base de données via une application

Introduction

Dans le paysage technologique actuel, la gestion des données est devenue le pilier central de toute application d'entreprise. Traditionnellement, l'administration d'une base de données relationnelle exigeait des efforts considérables en termes de provisionnement de matériel, d'installation de logiciels, de configuration de correctifs de sécurité et de mise en place de stratégies complexes de haute disponibilité. L'avènement du Cloud Computing, et plus précisément de services managés comme **Amazon Relational Database Service (RDS)**, a radicalement transformé cette approche en automatisant les tâches administratives chronophages.

Ce laboratoire pratique s'inscrit dans une démarche de compréhension approfondie de l'infrastructure Cloud. L'objectif principal est de démontrer comment lever les barrières techniques liées au déploiement d'une base de données robuste et évolutive. Nous explorons ici le concept de **Haute Disponibilité (Multi-AZ)**, une architecture où les données sont répliquées de manière synchrone sur plusieurs zones de disponibilité pour garantir la continuité de service en cas de panne matérielle ou de centre de données. À travers ce TP, nous ne nous contentons pas de créer un stockage de données ; nous construisons un pont sécurisé entre une couche applicative (serveur web EC2) et une couche de données persistante, tout en respectant les meilleures pratiques de sécurité réseau via les groupes de sécurité (Security Groups) et les groupes de sous-réseaux (Subnet Groups). Cette expérience simule un environnement de production réel où la séparation des responsabilités et la résilience sont primordiales.

Tâche 1 : Création d'un groupe de sécurité pour l'instance de base de données RDS

La sécurité dans AWS repose sur le modèle de "défense en profondeur". Avant même de déployer la base de données, il est impératif de définir un périmètre réseau strict. L'objectif de cette tâche était de créer un pare-feu virtuel, appelé **Groupe de Sécurité (Security Group)**, dédié exclusivement à l'instance RDS.

Analyse de la configuration :

1. **Isolation des ressources :** En créant le `DB Security Group` au sein du `Lab VPC`, nous nous sommes assurés que la base de données ne soit pas exposée publiquement.
2. **Principe du moindre privilège :** Au lieu d'ouvrir le port 3306 à tout le trafic Internet (ce qui serait une vulnérabilité critique), nous avons configuré une règle d'entrée (Inbound Rule) autorisant uniquement le trafic en provenance du **Web Security Group**.
3. **Chaînage de sécurité :** Cette méthode est une bonne pratique essentielle dans le cloud. En utilisant le nom du groupe de sécurité comme source (au lieu d'une adresse IP fixe), nous permettons au groupe de sécurité de la base de données de "reconnaître" dynamiquement tout serveur web qui serait associé au `Web Security Group`.

Résumé technique de l'implémentation :

- **Nom :** `DB Security Group`
- **Description :** Permit access from Web Security Group
- **Règle d'entrée (Inbound Rule) :**
    - **Type :** MySQL/Aurora
    - **Protocole/Port :** TCP / 3306
    - **Source :** `Web Security Group` (Référence ID)

Cette configuration garantit que si votre serveur web est compromis, l'attaquant ne pourra pas accéder à la base de données sans posséder les droits spécifiques liés au groupe de sécurité, et surtout, qu'aucune ressource externe à votre VPC ne pourra tenter de se connecter à votre instance MySQL, sécurisant ainsi l'accès aux données sensibles de l'application.

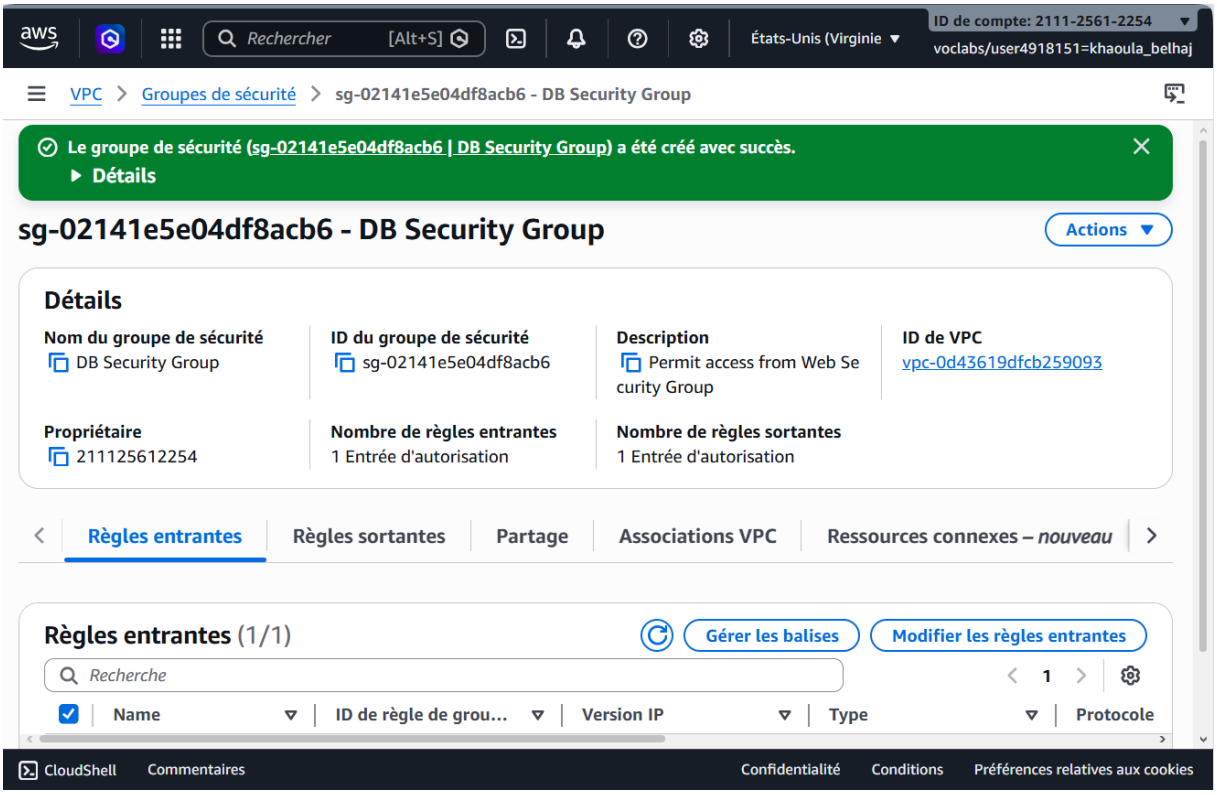


Figure 1 Créer un groupe de sécurité

Tâche 2 : Création d'un groupe de sous-réseaux de base de données

Après avoir défini les règles de sécurité à la tâche précédente, l'étape suivante a consisté à préparer l'infrastructure réseau physique (au niveau du Cloud) pour accueillir l'instance RDS. La création d'un **DB Subnet Group** est une condition préalable indispensable pour déployer une base de données Amazon RDS dans un VPC.

Analyse de la configuration :

1. **Définition du périmètre réseau (VPC) :** Le groupe de sous-réseaux a été rattaché au **Lab VPC**. Cela garantit que la base de données réside dans le même réseau logique que le serveur web, facilitant la communication interne tout en restant isolée d'Internet.
2. **Stratégie Multi-AZ (Multi-Availability Zones) :** L'aspect le plus critique de cette tâche a été la sélection de deux zones de disponibilité distinctes : **us-east-1a** et **us-east-1b**.
    - En informatique cloud, une "Zone de Disponibilité" correspond à un ou plusieurs centres de données physiquement isolés.
    - En associant des sous-réseaux de deux zones différentes (CIDR `10.0.1.0/24` et `10.0.3.0/24`), nous permettons à AWS de provisionner une instance de secours (standby) dans une zone si l'instance principale dans l'autre zone subit une défaillance.
3. **Gestion de la redondance :** Un groupe de sous-réseaux RDS doit toujours contenir au moins deux sous-réseaux dans deux zones de disponibilité différentes pour supporter la haute disponibilité. Cette configuration est ce qui permet à RDS d'assurer un basculement (failover) automatique sans intervention manuelle et sans changement de l'adresse de connexion (Endpoint) pour l'application.

Résumé technique de l'implémentation :

- **Nom du groupe :** `DB-Subnet-Group`
- **VPC cible :** `Lab VPC`
- **Zones de disponibilité sélectionnées :** `us-east-1a` et `us-east-1b`
- **Sous-réseaux inclus :**
  - Sous-réseau 1 : `10.0.1.0/24`
  - Sous-réseau 2 : `10.0.3.0/24`

Cette étape technique est la fondation de la résilience de notre base de données. Sans ce groupe de sous-réseaux correctement configuré, il serait impossible de garantir que les données restent accessibles en cas de sinistre majeur affectant un centre de données entier d'Amazon.

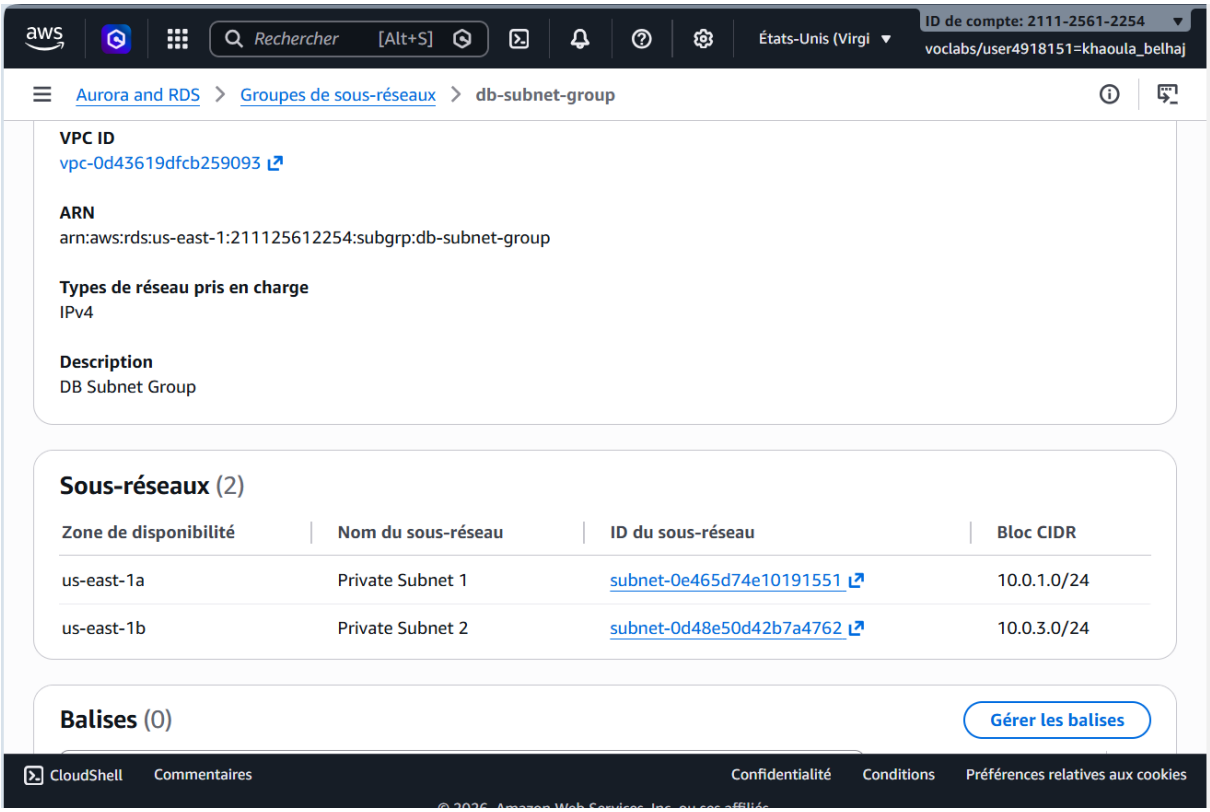


Figure 2 Créer un groupe de sous-réseaux DB

Tâche 3 : Création d'une instance de base de données Amazon RDS

Après avoir sécurisé l'accès réseau et défini les zones de disponibilité, l'étape finale de l'infrastructure consistait à lancer l'instance de base de données elle-même. Contrairement à une installation manuelle sur un serveur EC2, ici, nous avons orchestré le déploiement via l'interface gérée par Amazon RDS.

Analyse de la configuration :

1. **Choix du Moteur et Template :** Nous avons sélectionné **MySQL** avec le template **Dev/Test**. Ce choix optimise les ressources pour les environnements de développement tout en permettant de tester les fonctionnalités de haute disponibilité.
2. **Architecture Multi-AZ :** L'activation du mode **Multi-AZ** est l'élément le plus significatif. Ce choix déclenche la création automatique d'une instance primaire et d'une instance de secours (standby) dans une zone distincte. La réplication des données est gérée de manière synchrone par AWS, garantissant qu'en cas de défaillance, le basculement est quasi immédiat.
3. **Gestion des paramètres d'instance :**
    - **Instance Class (`db.t3.micro`) :** Choix optimisé pour les charges de travail burstables (évolutives).
    - **Stockage (20 Go SSD) :** Allocation suffisante pour la persistance des données dans le cadre de ce laboratoire.
    - **Sécurité et Réseau :** L'instance a été associée au `DB Security Group` (pour le filtrage) et déployée au sein du `DB-Subnet-Group` (pour la répartition géographique dans le VPC).

Résumé technique de l'implémentation :

- **Identifiant :** `lab-db`
- **Moteur :** MySQL
- **Mode de déploiement :** Multi-AZ
- **Nom de la base initiale :** `lab`
- **Point de terminaison (Endpoint) :** `lab-db.ctc8mscq2bt1.us-east-1.rds.amazonaws.com` (Récupéré après le temps de provisionnement de 4 minutes).

La phase de déploiement s'est conclue par l'attente du passage de l'état de l'instance de "Creating" à "Available".Le **Endpoint** récupéré est le lien critique qui permet à l'application web de s'identifier et de communiquer avec la base de données. Sans cette adresse, aucune couche applicative ne pourrait adresser ses requêtes SQL à notre serveur managé.

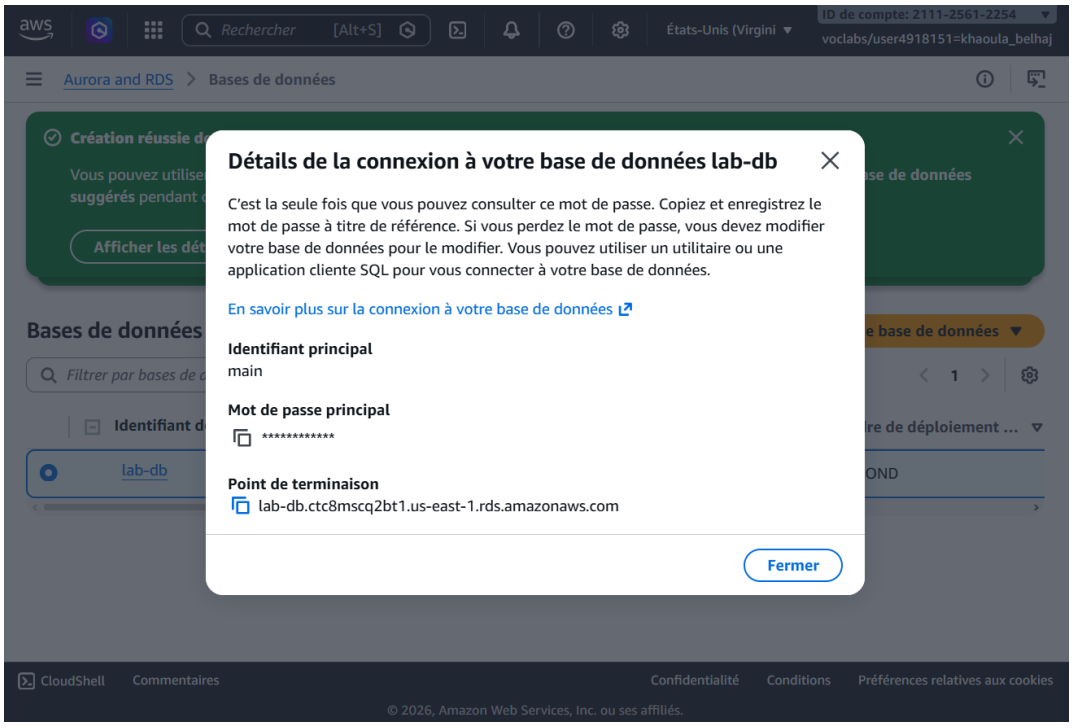


Figure 3 Créer une base de données

Tâche 4 : Interaction avec votre base de données

Cette dernière étape a permis de vérifier concrètement l'interopérabilité entre le serveur d'application (EC2) et le serveur de données (RDS). L'objectif était de transformer une infrastructure brute en une plateforme de services fonctionnelle.

Analyse du processus d'intégration :

1. **Connexion au Serveur Web :** En utilisant l'adresse IP publique de l'instance **WebServer**, j'ai accédé à l'interface utilisateur de l'application. Cette étape confirme que le serveur web est opérationnel et accessible via le protocole HTTP.

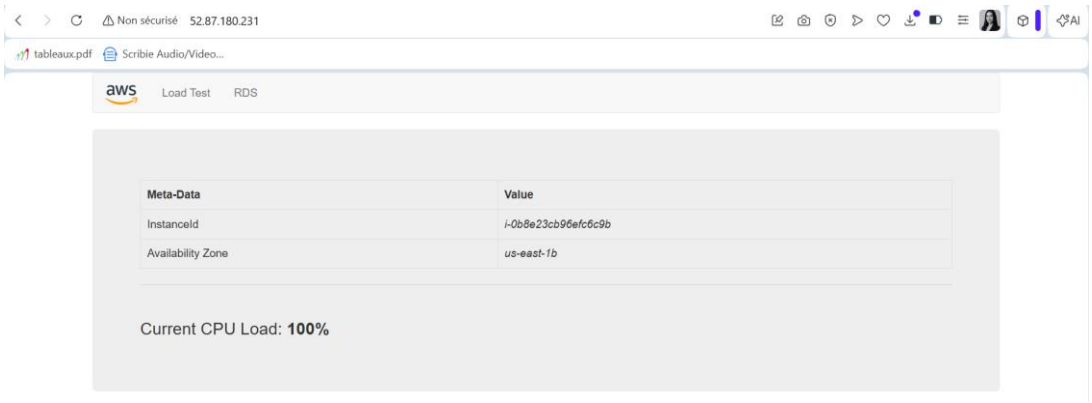


Figure 4 WebServer

2. **Configuration du lien Applicatif-Données :** L'application nécessitait des paramètres de connexion précis pour "pointer" vers la base de données. J'ai renseigné les informations suivantes :
    - **Endpoint :** L'adresse DNS spécifique fournie par RDS (le lien vital).
    - **Identifiants :** Le nom de la base (`lab`), l'utilisateur (`main`) et le mot de passe défini lors de la création.
3. **Validation de la persistance :** Une fois la connexion établie, l'application a initialisé les tables nécessaires dans MySQL. J'ai effectué des tests fonctionnels en ajoutant, modifiant et supprimant des contacts dans le "Address Book".

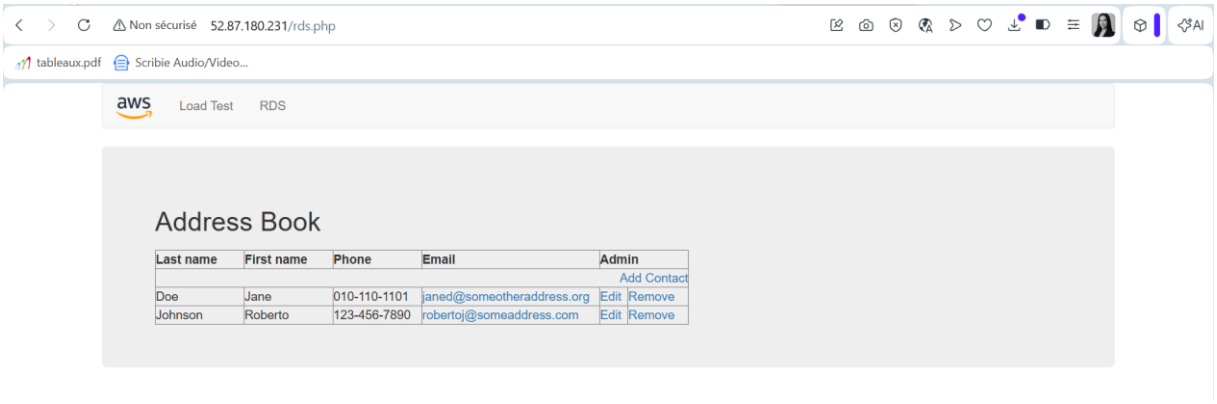


Figure 5 Address Book

Importance de la synchronisation :

Chaque action effectuée sur l'interface web déclenche une requête SQL vers l'instance RDS primaire. Grâce à la configuration **Multi-AZ** réalisée à la tâche 3, chaque donnée enregistrée est instantanément répliquée sur l'instance de secours. Ce test confirme que la chaîne de communication est fluide : **Client (Navigateur) → Serveur Web (EC2) → Base de données (RDS) → Réplication (Zone de secours).**

Résumé technique de l'exécution :

- **Action :** Liaison de l'application PHP/Python au serveur MySQL distant.
- **Résultat :** Affichage réussi du carnet d'adresses et persistance des données confirmée après rafraîchissement de la page.
- **Preuve de succès :** Les règles de flux définies dans le `DB Security Group` à la tâche 1 ont correctement laissé passer le trafic en provenance du serveur web tout en protégeant l'instance RDS.

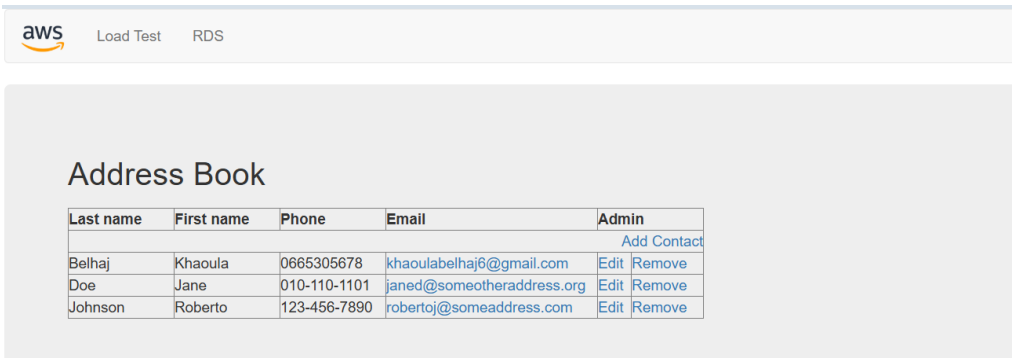


Figure 6 Add Contact

Conclusion

En conclusion, ce laboratoire a permis de concrétiser les concepts théoriques liés aux bases de données managées dans le cloud. La réussite de ce déploiement illustre parfaitement l'agilité qu'offre Amazon RDS : en moins de 30 minutes, nous avons mis en place une infrastructure qui, dans un centre de données traditionnel, aurait nécessité des jours de planification et de configuration matérielle. Nous avons pu observer que la complexité de la haute disponibilité est désormais masquée par une simple option de configuration "Multi-AZ", permettant aux développeurs de se concentrer sur l'innovation logicielle plutôt que sur la maintenance d'infrastructure.

L'aspect le plus formateur de ce TP réside dans la compréhension de l'interconnectivité des services AWS. La manipulation des **VPC**, des **Security Groups** et des **Subnet Groups** souligne l'importance d'une stratégie de défense en profondeur. Ce n'est pas seulement la base de données qui compte, mais la manière dont elle est intégrée de façon sécurisée et résiliente dans son écosystème réseau. Ce savoir-faire est indispensable pour tout architecte cloud souhaitant concevoir des systèmes capables de supporter des charges critiques tout en garantissant l'intégrité et la disponibilité constante des informations.

À l'issue de ce travail, les compétences acquises sur la gestion du cycle de vie d'une base de données RDS — de la création sécurisée à l'interaction applicative — constituent une base solide pour aborder des architectures plus complexes, telles que l'implémentation de réplicas de lecture pour la performance ou l'utilisation de bases de données NoSQL pour des besoins de scalabilité horizontale massive. Ce lab confirme que le passage au cloud n'est pas qu'une simple migration de serveur, mais un changement de paradigme vers une infrastructure programmable, fiable et hautement sécurisée.